SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical modelsincluding Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Assessment Tool Weightage: 40%

QUESTIONS: **Total Marks:25**

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, designa transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design adeep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

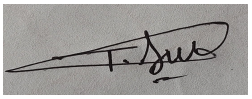
A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain howBackpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combinesTransfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the completesystem produces accurate captions.

Code of Conduct:

I T. Narasimha (192425233) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student 

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Answers:

Q1. Transfer Learning

Model Choice:

EfficientNet-B0 (or ResNet-50) pre-trained on ImageNet is the recommended backbone. ImageNet features (edges, textures, colour blobs, shapes) transfer well to dermoscopic images, and EfficientNet gives strong accuracy at low parameter/compute cost, which matters for a startup with limited labelled data and infrastructure.

Freezing / Fine-Tuning Strategy:

- Freeze the early convolutional blocks (low-level edge/texture filters) since these generic features are already well learned and do not need retraining on a small dataset.
- Unfreeze and fine-tune the last 2–3 convolutional blocks so the network can adapt mid/high-level features to lesion-specific patterns (asymmetry, border irregularity, colour variation).
- Replace the original classification head with a new fully-connected layer + softmax matching the skin-cancer classes (e.g., benign vs malignant, or multi-class lesion types).
- Train in two phases: first train only the new head with the backbone frozen (few epochs, higher learning rate), then unfreeze the top blocks and fine-tune the whole network at a much lower learning rate to avoid destroying pretrained weights.

Why Does This Improves Performance?

Transfer learning lets the model reuse millions of ImageNet images worth of visual knowledge, so it needs far fewer labelled dermoscopic images to converge. Freezing early layers prevents overfitting on the small dataset, while fine-tuning deeper layers lets the model specialise to lesion-specific texture and colour cues. Data augmentation (rotation, flipping, colour jitter) and class-weighted loss (since malignant cases are usually rarer) further improve generalisation and recall on malignant cases, which is the clinically critical metric.

Q2. DenseNet and PixelNet

Architecture Design:

Use a DenseNet (e.g., DenseNet-121) as the encoder/backbone and a PixelNet-style dense-prediction head as the decoder, forming an encoder–decoder segmentation network for the road scene.

- DenseNet Encoder: each layer receives feature maps from all preceding layers via dense connections, extracting rich multi-scale features for roads, pedestrians, vehicles, and traffic signs while reusing computation.
- PixelNet Decoder: takes hypercolumn features (concatenated feature maps from multiple DenseNet layers, upsampled to a common resolution) and passes them through per-pixel MLP classifiers to predict a class label for every pixel.
- Skip connections between encoder and decoder preserve fine spatial detail (thin objects like traffic signs and pedestrian edges) that would otherwise be lost during downsampling.

Feature Reuse, Accuracy and Efficiency:

- Feature reuse: dense connectivity means every layer accesses all previous feature maps, so features are learned once and reused everywhere, reducing redundant filters.
- Segmentation accuracy: pixel-level hypercolumn features combine low-level boundary information with high-level semantic context, giving sharper object boundaries for vehicles/pedestrians and better small-object detection for traffic signs.
- Computational efficiency: DenseNet needs fewer parameters than an equivalently deep plain CNN because of feature reuse, and this keeps the pipeline light enough for near-real-time road-scene analysis in a smart-city deployment.

Q3. Bidirectional RNN and Encoder–Decoder

Architecture:

Encoder: a Bidirectional RNN (Bi-LSTM/Bi-GRU) reads the input customer query in both forward and backward directions and concatenates the two hidden states at each time step, producing a context vector that captures information from the whole sentence, not just the words seen so far.

- Forward pass: processes the query left-to-right, capturing preceding context.
- Backward pass: processes the query right-to-left, capturing following context.
- The concatenated forward+backward hidden states form rich encoder representations, often combined with an attention mechanism so the decoder can focus on the most relevant source words at each output step.
- Decoder: a unidirectional RNN (LSTM/GRU) generates the translated/response sequence one token at a time, conditioned on the encoder's context vector (and attention weights) plus previously generated tokens.

Why Bidirectional Processing Improves Accuracy?

Customer queries often contain words whose meaning depends on later context (e.g., negation, idioms, or clauses that change intent). A unidirectional RNN only sees past words, so it can misinterpret such context. The bidirectional encoder sees the entire sentence before encoding, so ambiguous words are disambiguated using both preceding and following context. This produces a more complete semantic representation, which the decoder uses to generate translations that better preserve the original meaning and intent of the customer query.

Q4. BPTT and LSTM

LSTM-Based Prediction System:

The historical price series (open, high, low, close, volume, technical indicators) is normalised and split into fixed-length sliding windows. An LSTM network processes each window sequentially: at every time step the cell state and hidden state are updated using the input gate, forget gate, and output gate, allowing the network to retain relevant long-term price trends while discarding noise. The final hidden state is passed through a dense layer to predict the next day's (or next window's) price.

Role of BPTT:

- The LSTM is unrolled across all time steps of the input window, forming a deep computational graph in time.
- The loss (e.g., MSE between predicted and actual price) is computed at the output and gradients are propagated backward through every unrolled time step, updating the gate weights (input, forget, output, candidate) at each step.
- Gradients from later time steps flow back to earlier ones, so BPTT lets the network learn which past prices/patterns were relevant to today's movement.
- Truncated BPTT is typically used for long sequences, propagating gradients only a limited number of steps back to keep training computationally feasible.

Why LSTM over a Traditional RNN:

- Vanishing/exploding gradients: plain RNNs lose gradient signal over long sequences, so they cannot learn dependencies across many trading days. LSTM's gated cell state preserves gradient flow over long horizons.
- Long-term memory: stock trends (support/resistance levels, seasonal patterns) span long time windows; the LSTM's memory cell retains this information far better than a vanilla RNN's hidden state alone.
- Selective forgetting: the forget gate lets the model discard irrelevant old data (e.g., a one-off market shock) while keeping persistent trend information, improving forecast stability.

Q5. Integrated Deep Learning Solution

Integrated System Design:

The pipeline has two connected components: a Transfer-Learning-based visual encoder and an LSTM-based Encoder–Decoder caption generator.

- Visual Feature Extraction (Transfer Learning): a pre-trained CNN such as ResNet-50 or InceptionV3 (trained on ImageNet) is used, with its final classification layer removed. Video frames are sampled at fixed intervals and passed through this frozen/fine-tuned CNN to extract high-level visual feature vectors for each frame, avoiding the need to train a vision backbone from scratch on limited captioned video data.
- Encoder: the sequence of per-frame CNN feature vectors is fed into an LSTM encoder, which aggregates temporal visual information across the video into a single context representation (optionally with attention over frames).
- Decoder: a second LSTM (decoder) takes the encoder's context vector and generates the caption word by word, using an embedding layer for vocabulary and a softmax output at each step, conditioned on previously generated words (teacher forcing during training).

Role of Each Component and Why It Works:

- CNN (Transfer Learning): supplies strong, data-efficient visual features (objects, scenes, actions) without needing millions of labelled videos, since ImageNet pretraining already captures general visual patterns.
- LSTM Encoder: converts the variable-length sequence of frame features into a fixed context vector, capturing temporal/motion information that a single-frame CNN cannot.
- LSTM Decoder: generates grammatically coherent, contextually accurate captions by modelling word-to-word dependencies conditioned on the visual context.
- Combining transfer learning with the LSTM encoder-decoder means the system needs far less training data and compute than an end-to-end model, while still producing accurate, fluent captions that improve accessibility for the streaming platform's viewers.